

Data Engineer — Detailed Guide

Retail Demand Forecasting (Walmart M5, Hierarchical) · Team Study Material

1. What this topic is

Data engineering is the work of getting raw data into a shape a model can actually learn from — and proving that nothing was broken along the way.

In our project the data engineer owns everything that happens **before** any machine learning starts:

RAW FILES -> CHECK -> RESHAPE -> JOIN -> STORED DATASET -> modelling

Nobody trains a model on five separate spreadsheets. Somebody has to turn them into one clean table where each row means one thing. That is this job.

A word you will need: "row" and "column". A table is a grid. A **column** is a named field (for example sales, date, price). A **row** is one record — one combination of values across all those columns. In our final table, one row = "this product, in this store, on this day".

2. Why our project needs it

Our task is to predict how many units of each product each Walmart store will sell for the next 28 days. The competition data does **not** arrive in that shape. It arrives as three unrelated tables:

- one holding sales, with **one column per day** (1,941 day-columns);
- one holding the calendar, with events and SNAP flags;
- one holding weekly prices.

A model cannot read that. It needs one long table where every row carries the sales figure *and* the matching date, event and price. Producing that table — and proving it is faithful to the original — is the whole contribution of this role.

VERIFIED RESULT. *The reshaped and joined dataset contains **59,181,090 rows**. That is 30,490 store-item series x 1,941 days.*

3. The dataset

We use the public **M5 Forecasting — Accuracy** dataset (Walmart retail sales). It lives in `03_RESEARCH/data/raw/` and is treated as **immutable** — never opened in write mode by any code in this repository.

Property	Verified value
Series (store-item combinations)	30,490
Products (items)	3,049
Stores	10
States	3 (California, Texas, Wisconsin)
Departments	7
Categories	3
Days of sales history	1,941 (d_1 ... d_1941, 2011-01-29 → 2016-05-22)
Days in the calendar file	1,969 — 28 days *beyond* the sales, by design
Distinct price weeks	282
Total units sold across the whole dataset	66,927,173
Cells with zero sales	40,241,819 (68.00%)
Largest single-day sale	763 units

Those constants are not our estimates. They are asserted in `03_RESEARCH/pipeline/config.py` (`N_SERIES`, `N_HISTORY_DAYS`, `EXPECTED_TOTAL_UNITS`, `EXPECTED_ZERO_CELLS`, `EXPECTED_MAX_SALES`) and checked against the files every time the pipeline runs.

***The single most important fact about this data: 68% of it is zero.** Most products do not sell every day in every store. This one property drives almost every later decision — the choice of model, the choice of error metric, and how we present accuracy.*

The five raw files

File	Size	What it holds
<code>sales_train_evaluation.csv</code>	121.7 MB	30,490 rows; identifier columns plus <code>d_1...d_1941</code>
<code>sales_train_validation.csv</code>	120.0 MB	the same data ending 28 days earlier
<code>calendar.csv</code>	0.1 MB	one row per day: date, weekday, month, year, events, SNAP flags
<code>sell_prices.csv</code>	203.4 MB	one row per store-item- week , with the price
<code>sample_submission.csv</code>	5.2 MB	the required output layout (F1...F28)

***CSV — Comma-Separated Values.** A plain-text table where each line is a row and commas separate the columns. It is readable by anything, which is why data is distributed this way — but it is slow and bulky for 59 million rows, which is why we convert it (Section 7).*

SNAP

`calendar.csv` has three columns `snap_CA`, `snap_TX`, `snap_WI`.

SNAP = Supplemental Nutrition Assistance Program, a United States food-assistance benefit. The flag is 1 on days when the benefit could be spent in that state and 0 otherwise. Food sales rise on SNAP days, so it is a genuinely useful signal.

(This expansion is stated in our own project materials — it is not an outside guess.)

4. Core concepts, from the beginning

Wide format vs long format. The sales file is **wide**: one row per series, and 1,941 separate columns for days.

WIDE	id	d_1	d_2	d_3	...	d_1941
	FOODS_3_090	0	2	1	...	4

A model needs **long** format: one row per observation, with the day as a value rather than a column name.

LONG	id	day_idx	sales
	FOODS_3_090	0	0
	FOODS_3_090	1	2
	FOODS_3_090	2	1

Turning one into the other is called **melting** or **unpivoting**. One wide row of 1,941 day-columns becomes 1,941 long rows.

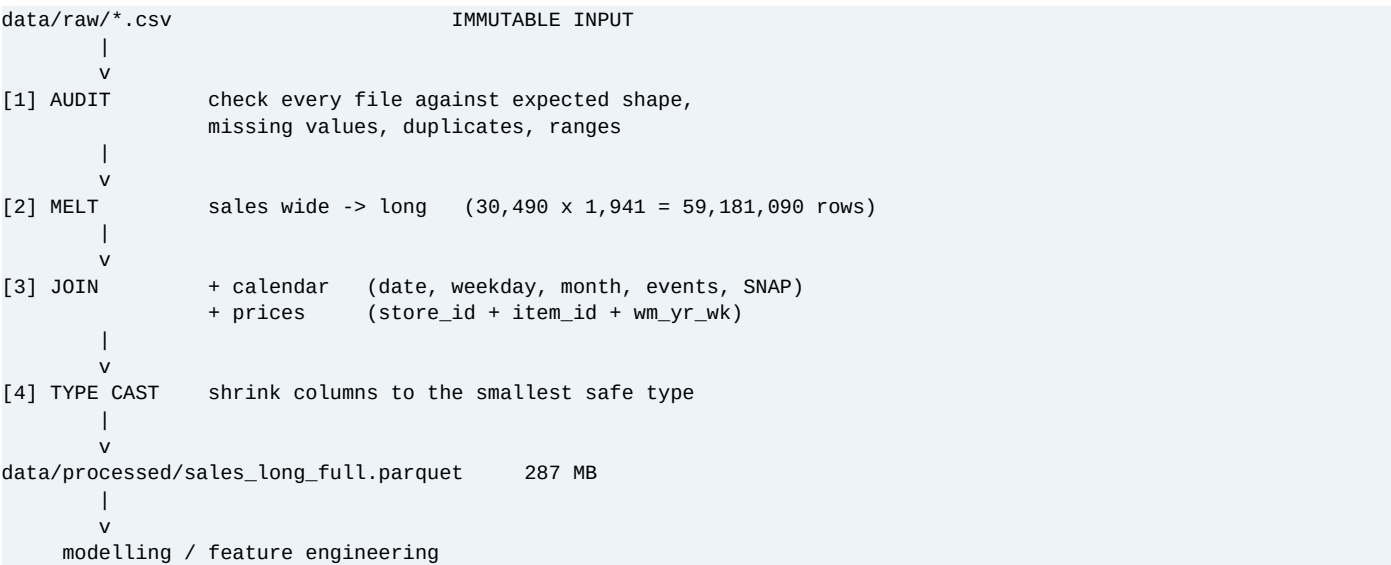
Joining. A **join** attaches columns from one table to another by matching on a shared key. To attach the date and the event to each sales row we join on the day identifier; to attach the price we join on store + item + week.

Key. *The column (or set of columns) that identifies a row uniquely. In `sell_prices.csv` the key is `store_id + item_id + wm_yr_wk`.*

Missing value / null. A cell with no value. Different from zero: zero means "we know it was none"; null means "we do not know". That distinction matters enormously here — see Section 6.

Data type. What kind of value a column holds — whole number (integer), decimal (float), text (string). Choosing small types saves a lot of memory when there are 59 million rows.

5. What we actually did — the pipeline



Every step is reproducible from the scripts in 03_RESEARCH/scripts/, and the paths are resolved centrally in 03_RESEARCH/pipeline/config.py so no script hard-codes a location.

6. What we found in the data

This is the part worth remembering, because the honest answer surprised us.

The data was already clean

We ran a full audit against the raw CSVs before doing anything. Checks covered missing values, duplicate identifiers, negative sales, calendar continuity, event-name/event-type consistency, SNAP values, price validity, duplicate price rows, and cross-table agreement between sales, calendar and prices.

VERIFIED RESULT — every one of these came back zero:

Check	Result
Missing sales values	0
Negative sales values	0
Duplicate sales id rows	0
Missing / duplicate calendar dates	0
Date-range gaps	0
event_name / event_type mismatches	0
Missing prices	0
Negative or zero prices	0
Duplicate store-item-week price rows	0
d_* columns absent from the calendar	0
Store / item ID mismatch between tables	0

So we fixed nothing. Per our own rule, nothing was "cleaned" for the sake of looking busy, because no genuine defect existed. That is a legitimate and defensible data-engineering outcome — and it is much stronger than inventing repairs.

The one thing that looks like a problem and is not

After joining prices onto the long sales table, **12,299,413 rows (20.78%)** have no price.

That is **not** missing data. A store-item simply has no price record before that product was introduced to that store's range. The absence is information: it tells us the product was not on sale yet. We therefore keep it as a null and let the feature layer represent it explicitly, rather than filling it in with a made up number.

Why this matters for the presentation. "20% of prices are missing" sounds like a data-quality failure. It is not. Explaining **why* it is expected is a strong answer.*

The zeros

68.00% of all sales cells are zero. We did **not** remove them. A zero is a real observation — "this product did not sell today" — and deleting zeros would teach the model that everything sells constantly. Keeping them is what makes this an **intermittent demand** problem, which is the defining characteristic of our dataset.

7. Storage — why Parquet, and then DuckDB

The processed dataset is written as `sales_long_full.parquet` (287 MB).

Parquet. A file format that stores data by **column** rather than by row, and compresses each column. For a table this size it is dramatically smaller and faster to read than CSV, and it remembers the data types, which CSV does not.

For the running product we build a second, smaller layer — this is a separate step and worth understanding as its own thing:

```
03_RESEARCH artefacts (read-only)
  |
  v
01_PROJECT/backend/scripts/build_product_db.py
  |
  v
01_PROJECT/backend/data/
  product.duckdb      small relational tables
  history.parquet     59.2M rows of actuals, sorted by series
  backtest.parquet    6.8M rows of backtest results
```

DuckDB. A database that runs inside the application rather than as a separate server. Think "SQLite for analytics". It lets the backend answer questions like "give me this product's history" in milliseconds.

Why a second copy at all? Because the research parquet repeats the item and store identifiers on all 59.2 million rows, and because a database view over an external file records an absolute file path — which breaks the moment the application runs on a different machine. Keeping only what the product needs gives **31.7 MB instead of 287 MB** and a single-series lookup in **8–13 ms instead of ~320 ms**. That reasoning is written up in the header of `build_product_db.py`.

8. Files you should be able to name

Path	Role
03_RESEARCH/data/raw/	the five original CSVs — immutable
03_RESEARCH/data/processed/sales_long_full.parquet	the joined long table, 59.2M rows
03_RESEARCH/data/processed/PROCESSING_REPORT.md	what was checked, found, fixed and deliberately not fixed
03_RESEARCH/data/processed/_audit/*.json	raw output of the audit checks
03_RESEARCH/pipeline/config.py	every path and every dataset constant, in one place
03_RESEARCH/scripts/01_foundation/01_foundation_check.py	the integrity and leakage suite
01_PROJECT/backend/scripts/build_product_db.py	builds the product data layer the API serves

9. Verification — how we know the data is intact

Three independent mechanisms, all of which actually run:

1. Checksums. Each raw CSV has a recorded MD5 checksum.

***MD5 checksum.** A short fingerprint computed from a file's contents. Change a single byte and the fingerprint changes completely. Comparing fingerprints proves a file was not modified.*

2. The foundation check. `01_foundation_check.py` runs **46 integrity and leakage checks** — shapes, totals, ranges, cross-table agreement, and whether any future information could leak backwards into a feature.

3. The artefact manifest. `03_RESEARCH/scripts/08_organization/61_integrity_manifest.py` records the SHA-256 hash of every protected file and compares runs. Its current state covers **522 protected files**, and the last comparison reported **0 deleted, 0 modified**.

***Hash / SHA-256.** Same idea as MD5, a newer and stronger fingerprint.*

10. Common misunderstandings

"Cleaning means removing rows." No. Cleaning means making the data **correct and consistent**. Here, correctness required removing nothing.

"Zeros are missing data." No. A zero is a measurement. A null is an absence. Our 68% zeros are real observations of no sale.

"We should fill the missing prices with the average." No. The absence marks a product that had not been introduced yet. Filling it in would invent a history that never happened.

"More data is always better." We deliberately ship a **smaller** derived dataset to the product (31.7 MB rather than 287 MB) because the product only needs certain columns and needs them fast.

11. How to present this

Say something close to this, in your own words:

"My part is the data. We used the Walmart M5 dataset — 30,490 product-store combinations over 1,941 days, which is about 59 million rows once you reshape it. The raw data arrives as three separate tables: sales with one column per day, a calendar, and weekly prices. My job was to turn that into a single long table where one row means one product, in one store, on one day, with its date, event and price attached.

Before touching anything I audited the raw files — missing values, duplicates, negative sales, calendar gaps, price validity, and whether the tables agree with each other. Everything came back clean, so we deliberately fixed nothing rather than invent repairs.

The two findings that matter: 68% of all cells are zero, because most products don't sell every day — that's what makes this an intermittent demand problem. And about 20% of rows have no price, which isn't a defect: those products simply weren't stocked in that store yet, so we keep the gap as information instead of filling it in.

Everything is verified by checksums and a 46-check integrity suite, and we track SHA-256 hashes over 522 protected files so we can prove nothing changed."

12. Likely judge questions

Beginner

Q: Where did the data come from? A: The public M5 Forecasting — Accuracy dataset from Walmart. It is in `03_RESEARCH/data/raw/` and we never modify it.

Q: How big is it? A: 30,490 series over 1,941 days — 59,181,090 rows once reshaped, about 66.9 million units sold in total.

Q: What does one row mean? A: One product, in one store, on one day: the units sold, plus that day's calendar information and the product's price that week.

Technical

Q: What cleaning did you do? A: Honestly, none was needed. We ran a full audit — missing values, duplicates, negatives, calendar continuity, price validity, cross-table key agreement — and every check returned zero problems. We documented that rather than manufacturing fixes.

Q: 20% of your prices are missing. Isn't that a problem? A: It is expected, not missing. A store-item has no price row before that product enters the store's range. We keep it null and let the model see "no price yet" as a signal, rather than imputing a value that never existed.

Q: Why didn't you drop the zero rows? They're 68% of the data. A: Because a zero is a real observation. Dropping them would train the model on a world where everything sells every day, and it would destroy exactly the behaviour we're trying to forecast.

Q: How do you know the data hasn't been altered? A: MD5 checksums on the raw files, a 46-check foundation suite, and a SHA-256 manifest over 522 protected files that we re-run and compare. The last run reported zero deleted and zero modified.

Q: Why Parquet and not CSV? A: Column storage plus compression, and it preserves data types. For the product we go further and build a DuckDB layer plus sorted sidecar files — 31.7 MB instead of 287 MB, and a single-series query in 8–13 ms instead of about 320 ms.

13. Glossary

Term	Meaning
CSV	Comma-Separated Values — plain-text table format
Parquet	Compressed, column-oriented file format; keeps data types
DuckDB	An analytics database that runs inside the application, no server
Row / column	One record / one named field
Wide vs long	One column per day vs one row per day
Melt / unpivot	Turning wide format into long format
Join	Attaching columns from another table by matching a key
Key	The column(s) that identify a row uniquely
Null	No value recorded — different from zero
Data type	Integer, float, string; smaller types save memory
MD5 / SHA-256	File fingerprints used to prove nothing changed
SNAP	Supplemental Nutrition Assistance Program — US food benefit; a calendar flag in our data
Intermittent demand	Demand that is zero on most days
Immutable	Never written to; read-only by rule